A validated regrouping of PA196 and its arithmetic transfer

Technical note

13 November 2026

Abstract

This note records a new 196-working-state layout of the unchanged PA196 register program. Regrouping the binary program tree saves one reader state, but its longest zero run becomes seven bits, which requires an eighth-zero return test and consumes that saving. The result passes all 4,959 uniform symbolic instruction contracts at minimum gap 48, and reaches the exact baseline clean frame from blank tape after 59,164 steps. It is a verified intermediate construction, not a machine below 196 states. We explain what the symbolic certificates establish, how their arithmetic proofs transfer the halting statement, and which steps have not been kernel-checked.

1 Pinned artifact and state convention

The artifact is `data/variant196.native.tm`. It has 196 working controls and 392 rows, starts at native control 74 on an all-zero two-way infinite binary tape, and uses the separate halt label 254. The native labels are sparse; they do not determine the state count by their maximum. Its SHA-256 digest is

`f6594edc95efed02f26b88f71c9b1f7699965a258739836f24e6b641887b0cd9`.

The matching source is `data/variant196_source.json`. Its SHA-256 digest is

`f2fd582615c47a680c6475235139d8026bd4126b03e788e7d18c76c1ff45c49e`.

A finite bijection can rename the 196 working controls to $0, \ldots, 195$, send native start 74 to dense start 0, and send the separate halt to 196. Checking the bijection against all rows proves literal trace equivalence; no state is removed by that operation.

The baseline is James Chang's PA196 at commit `407cdc356f6b586635e71ac92be394255f4bfea4`. The local baseline native table has SHA-256

`454254a42a9f9096b72f73ef491107713002ca32f2a51f165eee7ca299283215`.

This native-table digest differs from that of the upstream dense `data/pa196.tm` because the labels and serialization differ.

2 What changed

The 3,812 primitive source instructions retain their ordinal order, opcodes, register arguments and resolved successors. The axiom command emits the same formulas in the same order. Only binary parenthesization and physical sharing of the reader states change. Two independent successor

resolvers, one using binary address arithmetic and one using subtree intervals, give the same control-flow graph as the baseline, including both outcomes of conditional decrement.

The regrouped source has 158 distinct internal binary nodes, one fewer than the baseline's 159. It has 23 distinct instruction leaves; thus the runtime checker's reported total of 181 distinct DAG nodes includes leaves and is consistent with the internal-node count. Six selected context folds in the regrouped AX region share complementary reader occurrences; inherited sharing outside this region is retained. These are contextual equivalences, not ordinary merges of identical transition rows. Their correctness is included in the complete instruction contracts below.

The maximum source-path depth is 36 and the maximum consecutive zero run is 7. A seven-zero return scanner can mistake such an internal path run for the blank exterior and resume at the wrong path position. Hence the nominal 195-state layout is not accepted. The repaired table inserts the following eighth test, in native labels:

	read 0	read 1
34	$(0, L, 2000)$	$(1, L, 2)$
2000	$(0, L, 75)$	$(1, L, 2)$.

The preceding zero-count states are $2, 3, 5, 9, 15, 23, 34$. On a one they reset leftward to state 2. The extra state 2000 distinguishes a path run of seven zeros from the longer blank run used for returning to the root. This costs one working state, bringing the repaired table back to 196. A static maximum-zero count is useful for the design, but the full physical contracts, rather than that count alone, validate the repair.

3 The checked results

The receipts `certificates/variant196/runtime.json` and `certificates/variant196/startup.json` record:

Quantity	Result
Working states / table rows	196 / 392
Source sites / decrement sites	3,812 / 1,147
Uniform instruction cases	4,959
Rows covered by the symbolic contracts	392
Minimum permitted header gap	48
Maximum source-path depth / zero run	36 / 7
Symbolic inference steps	1,177,411
Smallest constant term in a contract duration	3
Blank-start steps to the baseline clean frame	59,164

The 4,959 cases consist of one case per source site plus the second outcome at each decrement site. Every case closes for nine independent arbitrary natural register values, an arbitrary first spare value, an arbitrary finite remaining spare list, and every gap $48 + g$ with $g \in \mathbb{N}$. This is not a finite sampling of small register contents.

Startup is a separate exact finite simulation from the blank tape. After 59,164 steps the new table has control 84, head position -2, and exactly the same 66 one-cells as the baseline after its 58,154-step startup. The first divergence in their literal runs is at step 29, so the startup claim is equality of the final clean frame, not equality of the two complete traces. The initial source site 25 retains path `100000`, so the common physical frame represents the same initial source configuration. Its gap is 48, within the uniformly checked range.

4 Soundness of the symbolic certificate rules

A symbolic tape contains finite known bits, runs whose lengths are affine expressions in independent natural parameters, a finite list of spare register blocks, and the infinite blank exterior. The checker accepts a literal read from a variable run only when its length is known to be positive, or handles the whole run by an explicit self-loop induction. A proof that needs to inspect an unrepresented or possibly empty region is rejected.

The elementary proof rules are as follows.

1. A literal transition is the corresponding row of the supplied machine table and changes precisely one scanned cell and the head.

2. A unary self-loop traverses a whole run. If its row is (q, b, w, D, q), induction on n proves its exact action on b^n, including $n = 0$, its duration n, and its displacement nD.

3. The four generic one-register traversals are rechecked from the actual table. They act on $B(v) = 0\,1^{v+1}$ or $C(v) = 1\,0\,1^v$, have duration $v + 2$, and retain opaque contexts on both sides. Empty/cons induction extends each one-block theorem to every finite spare list.

4. Adjacent equal-bit runs combine by addition of their lengths. Reversal, the identities $C(A)1 = 1B(A)$ for lists of blocks and their reversed counterparts, and absorption of finite zero runs into the blank exterior preserve the represented tape. These are direct word identities, proved by finite-list induction where necessary.

Composition of these steps proves every complete contract. The reported durations have nonnegative affine coefficients and positive constant terms, ensuring genuine positive physical progress. The target configurations show exact register effects and successor paths, with nondecreasing gap and the required spare-file invariant. The four generic block lemmas and all site contracts are rederived from the new table; no unchanged name of an engine state is used as an unverified theorem.

All the above proofs are ordinary inductions on finite encodings. They are formalizable in PA, including for nonstandard parameter values in a PA model. This mathematical soundness argument, together with successful certificate replay, justifies the uniform contracts. The Python checker is part of the trusted proof boundary. Its successful execution is not being described as a Lean or Rocq kernel check.

An explicit bound for the arithmetic trace construction. The affine duration vectors give a concrete bound on the PA witness construction. At every live boundary after n primitive source steps from the checked clean seed, the header-gap excess over 48 is at most n, the sum of the nine register values is at most $n+2$, the first spare remains 1, and the width of the remaining spare list is at most $75+3n$. The initial spare file has 26 entries of value 1, and each block $B(1) = 011$ has width 3. Every increment appends one such block and raises one register by one; every positive decrement increases the gap by one and lowers one register by one; the other branches preserve these quantities. All source-path increments and cuts satisfy the checked no-root-overflow condition. Without that condition, an additional header-displacement term would have to be included; the present estimate does not silently drop such a term.

Write a checked duration vector as $(a_0, a_g, a_1, \ldots, a_9, a_s, a_w)$ and put $m = \max(a_1, \ldots, a_9)$. Nonnegative coefficients and the preceding invariants bound this primitive's duration by

$$(a_0 + 2m + a_s + 75a_w) + (a_g + m + 3a_w)n.$$

In a positive-decrement case, the tested register parameter represents its value minus one, so this estimate remains valid. In a zero-decrement case its unused parameter has coefficient zero, which is checked explicitly. The standalone `code/check_trace_bound.py` checks all 4,959 vectors, their complete case census, the full canonical seed, and every relevant no-overflow source path. The maximum constant is 700 and the maximum slope is 18. Hence a trace implementing a live prefix of T primitive source steps, or one whose final instruction halts, has length, including blank startup, at most

$$\boxed{59{,}164 + 700T + 9T(T-1)}.$$

This is the exact sum of $700 + 18n$ for $0 \le n < T$, plus startup. It gives an elementary PA-provable bound for producing the literal trace witness. The conclusion relies on the established uniform runtime contracts; checking these numerical inequalities is not a replacement for their symbolic induction proofs and does not imply nonhalting by bounded simulation.

5 Arithmetic transfer

Let R denote this repaired table, M_{196} the baseline, and S their identical ordinal source program. Let H_R, H_{196}, H_S be the respective exact coded halting sentences.

Theorem 1. *The source identity, startup trace and uniform positive contracts, with the preceding PA-formalizable soundness proof, give*

$$\mathrm{PA} \vdash H_R \leftrightarrow H_S \leftrightarrow H_{196}.$$

Consequently the published internal consistency implication transfers:

$$\mathrm{PA} \vdash \neg H_R \to \mathrm{Con(PA)}.$$

With the direct uniform syntactic PA proof translation for the native calculus, described in the accompanying soundness note, the full paper-level conclusion is

$$\mathrm{PA} \vdash \neg H_R \leftrightarrow \mathrm{Con(PA)}.$$

Proof. A finite source halt gives a literal halt by prepending startup and concatenating its instruction contracts. Conversely, if a literal halt occurs at time T but the source does not halt, PA constructs $T+1$ source steps and concatenates their positive literal segments. The result is a genuine literal trace longer than T with a working endpoint. Determinism identifies its configuration at time T with the alleged halt, which has no outgoing transition. This contradiction proves the converse. Both arguments use PA induction on coded finite sequences, so they handle nonstandard lengths. Apply them to both tables to obtain the first equivalences.

The upstream development proves the internal direction $\neg H_{196} \to \mathrm{Con(PA)}$, which transfers immediately. For the reverse direction, the separate syntactic translation maps membership to binary digits and compiles every guarded separation instance into a PA proof using finite bit comprehension. It retains proof witnesses beside all live source formulas. Under the internal hypothesis Con(PA), Shift cannot reach the false code 1, so all command boundaries remain live. The positive command/source and source/tape contracts then give arbitrarily long live literal traces and exclude halting by the same determinism argument. □